Set_A_Lab6 - Results

Attempt 1 of 1

Written Feb 18, 2026 6:33 PM - Feb 18, 2026 7:03 PM

Attempt Feedback

30% penalty
Absent 18 feb
22/22

Attempt Score	**15.4 / 22 - 70 %**
Overall Grade (Highest Attempt)	**15.4 / 22 - 70 %**

Question 1

https://cocalc.com/

Consider the following c code, first compile your code to obtain the binary file main1.out and then run it.

main1.out

```c
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4
5  int main(){
6      printf("You will see this line number = %d.\n", __LINE__);
7      fflush(stdout);
8      static char *args[] = {"", "-a","-l", NULL};
9      execve("/usr/bin/ls", args, 0);
10     printf("You wont be able to see this line number (%d) .\n", __LINE__);
11     return 0;
12 }
13
```

Insert a screenshot of your output.

```
~$ gcc -o main1.out main1.c
~$ ls
2026-02-19-terminal-2.term  main1.c  main1.out
~$ ./main1.out
You will see this line number = 6.
total 17
drwxr-xr-x 5 user user    11 Feb 19 02:44 .
drwxr-xr-x 1 root root  4096 Feb 19 02:42 ..
lrwxrwxrwx 1 user user    18 Feb 19 02:42 .bash_profile -> /home/user/.bashrc
-rw-r--r-- 1 user user  2355 Feb 19 02:42 .bashrc
drwxr-xr-x 3 user user     3 Feb 19 02:42 .cache
lrwxrwxrwx 1 user user    12 Feb 19 02:42 .smc -> /tmp/.cocalc
dr-xr-xr-x 2 user user     2 Jan  1  1970 .snapshots
drwxr-xr-x 2 user user     3 Feb 19 02:42 .ssh
-rw-r--r-- 1 user user    28 Feb 19 02:44 2026-02-19-terminal-2.term
-rw-r--r-- 1 user user   332 Feb 19 02:43 main1.c
-rwxr-xr-x 1 user user 16168 Feb 19 02:44 main1.out
~$
```

Screenshot 2026-02-18 at 6.40.22 PM.png (140.58 KB)

Screenshot 2026-02-18 at 6.44.50 PM.png (112.83 KB)

The correct answer is not displayed for Written Response type questions.

Question 2

Referring to your output from **Question 1**, does line 10 appear in the output (Y/N)?

___N___ ✔

Question 3

What is the primary purpose of the `execve()` system call?

- To terminate the running process
- ✔ To replace the current process image with a new program
- To create a new child process
- To put the process to sleep

Question 4

What happens to the current process after a successful call to execve()?

- It forks a new process and returns to the caller
- It enters a waiting state
- It continues executing the next instruction
- ✔ It is completely replaced by the new program

Question 5

If execve() fails, what value does it return?

- It never returns a value
- ✔ -1
- 0

- 1

Question 6

Which statement correctly describes the return behavior of execve() on success?

- It returns 0 and continues executing the original program
- It returns 1 to indicate a new program is loaded
- ✔ It never returns because the current process image is replaced
- It returns a file descriptor for the new executable

Question 7

https://cocalc.com/
Consider the following c code, first compile your code to obtain the binary file main2.out and then run it.

main2.out

```c

#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <fcntl.h>

int main()
{
    system("rm -f myfile* first_link second_link");
    int fd = open("myfile.txt", 'a');
    int flink = link("myfile.txt", "first_link");
    system("ln -s myfile.txt second_link");
    printf("The links are created.\n");
    close(fd);
    return 0;
}
```

Insert a screenshot of your output.

```
~$ gcc -o main2.out main2.c
~$ ./main2.out
The links are created.
~$
```

Screenshot 2026-02-18 at 6.49.19 PM.png (15.84 KB)

The correct answer is not displayed for Written Response type questions.

Question 8

Referring to your output from Question 7, complete the following table.

	myfile.txt	first_link	second_link
Number of links	**N1**	**N2**	**N3**
type of link (hard/soft)	**N/A**	**T1**	**T2**

N1 = ? e.g. 4 ___2___ ✔(20 %)

N2 = ? e.g. 4 ___2___ ✔(20 %)

N3 = ? e.g. 4 ___1___ ✔(20 %)

T1 e.g. hard or soft ___hard___ ✔(20 %)

T2 e.g. hard or soft ___soft___ ✔(20 %)

Question 9

https://cocalc.com/

Examine the provided C code snippets. Initially, compile the code to generate the binary files named main31.out and main32.out. Execute the subsequent commands to generate a list of system calls utilized in each scenario.

case 1: strace –c ./main31.out	case 2: strace –c ./main32.out

main31.c
`void main() {`
 	
`}`
	main32.c
	`#include <stdio.h>`
	`void main(){`
	`  printf("Hello world \n");`
	`}`

Insert a screenshot of your output for each case.

```
~$ strace -c ./main31.out
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 48.30    0.000242         242         1           execve
 12.38    0.000062           7         8           mmap
  7.58    0.000038          19         2           openat
  7.39    0.000037          12         3           mprotect
  5.19    0.000026          26         1           munmap
  3.59    0.000018          18         1           prlimit64
  2.40    0.000012           6         2           close
  2.40    0.000012           6         2           fstat
  2.00    0.000010           5         2           pread64
  2.00    0.000010          10         1         1 access
  1.20    0.000006           6         1           read
  1.20    0.000006           6         1           set_tid_address
  1.20    0.000006           6         1           set_robust_list
  1.20    0.000006           6         1           rseq
  1.00    0.000005           5         1           brk
  1.00    0.000005           5         1           arch_prctl
------ ----------- ----------- --------- --------- ----------------
100.00    0.000501          17        29         1 total
~$ strace -c ./main32.out
Hello world
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
  0.00    0.000000           0         1           read
  0.00    0.000000           0         1           write
  0.00    0.000000           0         2           close
  0.00    0.000000           0         3           fstat
  0.00    0.000000           0         8           mmap
  0.00    0.000000           0         3           mprotect
  0.00    0.000000           0         1           munmap
  0.00    0.000000           0         3           brk
  0.00    0.000000           0         2           pread64
  0.00    0.000000           0         1         1 access
  0.00    0.000000           0         1           execve
  0.00    0.000000           0         1           arch_prctl
  0.00    0.000000           0         1           set_tid_address
  0.00    0.000000           0         2           openat
  0.00    0.000000           0         1           set_robust_list
  0.00    0.000000           0         1           prlimit64
  0.00    0.000000           0         1           getrandom
  0.00    0.000000           0         1           rseq
------ ----------- ----------- --------- --------- ----------------
100.00    0.000000           0        34         1 total
~$
```

Screenshot 2026-02-18 at 6.56.36 PM.png (184.85 KB)

The correct answer is not displayed for Written Response type questions.

Question 10

Referring to your output from **Question 9**,

Did you obtain identical outcomes in both scenarios (Y/N)? ___N___ ✔(33.33 %)

Which system call was responsible for displaying "Hello, world!" ? e.g. read

___write___ ✔(33.33 %)

Which file descriptor was utilized for displaying "Hello world!"? e.g. 5 ___1___ ✔ (33.33 %)

Question 11

https://www.onlinegdb.com/online_c_compiler

Mini Linux Terminal

Consider the following C program. Execute the commands below sequentially, one at a time. After running them, capture and insert a screenshot showing the output. Ensure that the output corresponding to each command is clearly visible in your screenshot.

1) date
2) whoami
3) ls -al
4) uname -a
5) exit

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 1024
#define MAX_ARGS 64

int main() {
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    while (1) {
        printf("COMP_4736> ");
        fflush(stdout);
        if (fgets(input, MAX_INPUT, stdin) == NULL) {
            break;
        }
        input[strcspn(input, "\n")] = '\0';
        if (strcmp(input, "exit") == 0) {
            break;
        }
        int i = 0;
        args[i] = strtok(input, " ");
        while (args[i] != NULL && i < MAX_ARGS - 1) {
            i++;
            args[i] = strtok(NULL, " ");
        }
        pid_t pid = fork();

        if (pid == 0) {
            execvp(args[0], args);
            perror("execvp failed");
            exit(EXIT_FAILURE);
        }
        else if (pid > 0) {
            wait(NULL);
        }
        else {
            perror("fork failed");
        }
    }
    return 0;
}
```

```
COMP_4736> date
Thu Feb 19 02:59:57 AM UTC 2026
COMP_4736> whoami
runner61
COMP_4736> ls -al
total 28
drwxrwxrwt 2 runner61 runner61    80 Feb 19 02:59 .
drwxr-xr-x 1 root     root      4096 Jan 13 01:34 ..
-rwxr-xr-x 1 runner61 runner61 16528 Feb 19 02:59 a.out
-rw-r--r-- 1 runner61 runner61   962 Feb 19 02:59 main.c
COMP_4736> uname -a
Linux Check 6.8.0-1045-gcp #48~22.04.1-Ubuntu SMP Tue Nov 25 13:07:56 UTC 2025 x86_64 x86_64 x86_64 GNU/Linux
COMP_4736> exit


...Program finished with exit code 0
Press ENTER to exit console.
```

Screenshot 2026-02-18 at 7.00.42 PM.png (122.47 KB)

The correct answer is not displayed for Written Response type questions.

Question 12

Referring to your output from **Question 11,** what is the main advantage of execvp() over execve()?

- ✔ (selected) It searches the PATH variable
- It runs faster
- It allocates memory
- It creates a new process

Question 13

Referring to your output from **Question 11, i**f execvp() executes successfully, it:

- Returns PID
- Returns 0

- ○ Returns 1
- ✔ ◉ Does not return

Question 14

Referring to your output from **Question 11,** why does the parent call wait()?

- ○ To restart the shell
- ○ To stop itself
- ○ To free memory
- ✔ ◉ To wait for child process termination

Question 15

Referring to your output from **Question 11**, what happens if fork() returns 0?

- ✔ ◉ Child process
- ○ Error occurred
- ○ Parent process
- ○ System crash

Question 16

File descriptor numbers are allocated by open() starting from:

- 1
- 0
- ✔ The lowest unused descriptor
- 2

Question 17

Under normal conditions, calling open() will not return 0, 1, or 2 because:

- ✔ They are already in use
- They are illegal values
- The kernel blocks them

Question 18

What does the directory /proc/<pid>/fd represent?

- File system mount points
- ✔ Open file descriptors of a process
- Running services
- Kernel modules

Question 19

Which statement is CORRECT regarding file descriptor allocation?

- ✔ open() returns the lowest unused descriptor
- Descriptor numbers are random
- open() returns the highest available descriptor
- open() always returns descriptor 3 or higher

Question 20

In Linux, file descriptors 0, 1, and 2 are:

- ✔ Automatically given to each process by the OS and can be seen in /proc/<pid>/fd
- Only used by the kernel internally and invisible to users
- Assigned only when a program opens files manually
- Reserved numbers that cannot appear in /proc/<pid>/fd

Done